Informe de práctica de la semana 3

Herencia polimorfismo y sustitución

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S3-001 · 19 de octubre de 2026 · Informe revisado

1 Propósito y actividades

Implementar una planilla heterogénea de personal, comprobar un contrato común y justificar las relaciones de herencia del proyecto hotelero.

Docente y Administrativo estaban incompletos. La jerarquía necesita preservar remuneración positiva y nombre válido en todos los subtipos. El ejemplo de Cuadrado se conserva como contraejemplo intencional y se ofrece una alternativa con Figura independiente.

Actividades desarrolladas

Actividad	Archivo o componente	Resultado
1 IS A y HAS A	ACTIVIDADES_RESUELTAS.md	Se justifican las seis relaciones solicitadas.
2 Planilla	python/solucion.py	Docente, Administrativo e Investigador con validación y despacho polimórfico.
3 Liskov	python/solucion.py	Contrato común comprobado; fallo esperado del cuadrado incompatible.
4 C++	cpp/solucion.hpp	Base abstracta, destructor virtual, override y vector de unique_ptr.
5 Hotel	python/hotel_herencia.py y cpp/hotel_herencia.hpp	Simple, Doble y Suite sustituyen a Habitacion al cotizar.

Caso explicado

La planilla recorre cuatro objetos de personal y les solicita su remuneración mediante el mismo contrato. Las implementaciones calculan S/ 1680, S/ 1620, S/ 3150 y S/ 3000; la suma es S/ 9450. El cliente no necesita preguntar si cada objeto es docente, administrativo o investigador para elegir una fórmula.

Conceptos para interpretar el caso

Polimorfismo significa que una misma operación se resuelve según el objeto concreto. El principio de sustitución de Liskov exige que un subtipo pueda usarse donde se espera su base sin romper el contrato que el cliente necesita.

La solución y sus pruebas se encuentran en desarrollo. Las respuestas completas y el notebook amplían los ejemplos de este informe.

2 Desarrollo en Python y C++

La planilla llama calcular_remuneracion o calcularRemuneracion a través del contrato base. No elige fórmulas mediante condicionales sobre el tipo del objeto. Los cuatro ejemplos producen 1680, 1620, 3150 y 3000; el total es 9450.

Se valida que el bono administrativo sea finito y no negativo, además de un salario positivo. Esa regla adicional evita construir una subclase que incumpla el contrato de remuneración positiva. Las horas admiten 1 a 40; las categorías usan los factores 1.0, 1.4 y 1.8.

La regla correcta de Liskov es no fortalecer precondiciones ni debilitar poscondiciones. La formulación inversa de un párrafo de la guía no se usa como regla de implementación. Cambiar las dos dimensiones de CuadradoIncompatible rompe la independencia prometida por Rectangulo.

Análisis de las seis relaciones

Relación analizada	Decisión	Justificación
Posgrado y Estudiante	IS A	El subtipo conserva el contrato de estudiante y añade especialización.
Sensor y Dispositivo	IS A	Un sensor es un dispositivo con una función específica.
Informe e Impresora	HAS A o dependencia de uso	El informe utiliza una impresora; no es una impresora.
LibroDigital y Libro	IS A con contrato común	Metadatos y lectura compartidos; no exigir soporte físico en la base.
Motocicleta y Vehiculo	IS A	Comparte el contrato de vehículo y lo especializa.
Departamento y Empleado	HAS A por agregación	Agrupa empleados cuya existencia no depende del departamento.

Cómo seguir la implementación

Python resuelve calcular_remuneracion() a partir del objeto que recorre la planilla. C++ llama a calcularRemuneracion() mediante un puntero a la base; virtual permite el despacho y override verifica la sobrescritura. unique_ptr administra la propiedad y el destructor virtual permite destruir correctamente los objetos a través de la base.

3 Verificación y resultados

Las pruebas registradas aprobaron 13 casos Python y 13 casos C++17. Se ejecutaron en Python 3.12.14 y g++ 10.3.0. Las demostraciones de ambos lenguajes producen la misma salida.

Comprobación	Resultado	Registro
Pruebas Python	13 aprobadas	evidencias/pruebas_python.txt
Pruebas C++17	13 aprobadas	evidencias/pruebas_cpp.txt
Compilación y demostraciones	Sin advertencias y salidas iguales	evidencias/verificacion.json

Pytest ejecutó los mismos casos que unittest; sus resultados no se suman como pruebas diferentes. Las compilaciones usaron -Wall, -Wextra, -Wpedantic y -Werror.

Casos representativos comprobados

Caso	Resultado observado	Qué demuestra
Cuatro objetos en la planilla	Total S/ 9450	Cada subtipo aporta su cálculo mediante el contrato común.
CuadradoIncompatible como Rectangulo	Fallo de contrato esperado y capturado	La herencia elegida no conserva la independencia entre dimensiones.
Rectangulo de 4 por 5 y Cuadrado de lado 5	Áreas 20 y 25	La alternativa corregida conserva operaciones coherentes.
Habitaciones simple doble y suite	Precios 100 150 y 200	Una misma consulta usa las tres implementaciones.

El fallo del cuadrado incompatible es un resultado esperado de la prueba del contraejemplo. No se oculta un error del programa: se demuestra por qué esa relación de herencia incumple el contrato y se comprueba una alternativa independiente.

Lectura de la salida

TOTAL=9450.00 es la suma de las cuatro remuneraciones y CONTRATOS=4 confirma que los cuatro objetos cumplen el contrato común. El contraejemplo de Liskov se comprueba por separado como fallo esperado.

4 Aplicación al hotel y conclusiones

La jerarquía hotelera mantiene una base Habitacion y tres variantes con precio_noche o precioNoche sobrescrito. Una colección heterogénea cotiza 100, 150 y 200 para una tarifa base de 100. La capacidad y los invariantes siguen perteneciendo a la base. En esta etapa, Reserva se documenta como una clase independiente y no hereda de Habitacion.

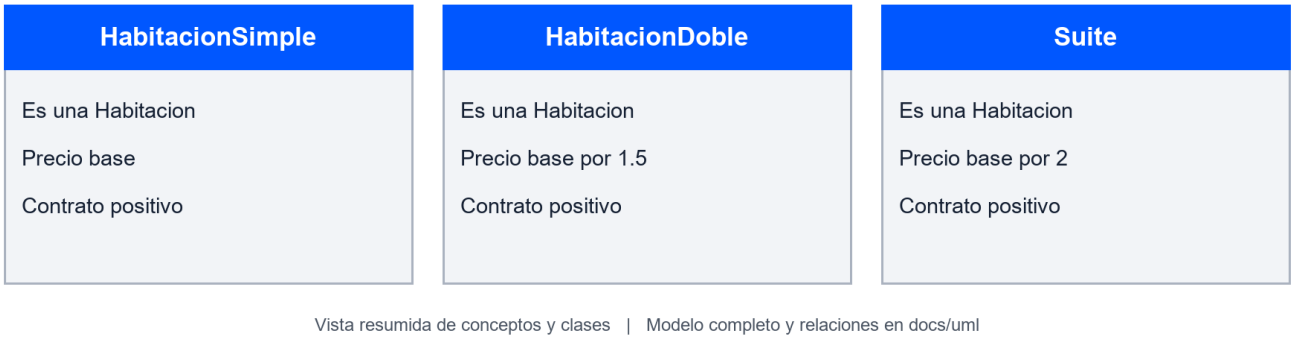


Figura 1 Esquema de los conceptos aplicados al hotel. El diagrama de clases completo se encuentra en docs/uml.

Conclusión de la práctica

La planilla y las habitaciones usan despacho polimórfico con resultados positivos y verificables. El contraejemplo muestra que la semejanza entre conceptos no basta para justificar herencia; la decisión depende de las operaciones que el cliente espera conservar.

Para comprobar la comprensión, seguir el caso explicado en los dos lenguajes e identificar la validación, el cambio de estado y la prueba que lo verifica. El anexo ACTIVIDADES_RESUELTAS.md desarrolla las preguntas de reflexión.

Transparencia y datos de entrega

Se utilizó Codex para analizar el material, desarrollar y verificar el código y preparar los documentos. Antes de entregar, completar los datos del estudiante y GitHub. La revisión de pares y la sustentación requieren participación humana.

Referencias

Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html

Object Management Group. (2017). *OMG Unified Modeling Language Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio de la semana 3*. SIS206 POO II, Universidad Nacional del Altiplano. Material del curso.